



Bits, Bytes & Nibbles? The hungry developers guide to binary

If you want to know how computers work under the hood and what's going on how the data is made up inside the memory you won't get around bits, bytes, and binary. In this post, I want to introduce you to and dive into how all of this works. We look into it from the start and explain everything. Including how tech people are dorks with naming things, and apparently hunger. Just look at some of the [obsolete units](#) that people used for it. It has such gems as:

- chomp (for multiple different units)
- bento box
- gobble
- plate

Thankfully those are obsolete things we don't have to worry about.

What are bits and bytes?

Bits and bytes are the basic underlying units of storage in a computer system. If you break things down everything in the end will be stored as bytes and therefore also bits.

To start off you can think of a bit like a light switch. It has two different states. On and Off . Electricity or no electricity. In the same way a `bit` has two distinct

values it can represent: On (1) or off (0). That also makes it the smallest unit of storage/information available in computers. But obviously, 2 distinct values aren't enough, we need a lot more options to represent the data we need day to day. We achieve this by combining multiple bits in the same way that we combine multiple digits in our decimal (base 10) numbers. 531 is just the combination of 1 in the ones column, 3 in the tens column, and 5 in the hundreds column.

A byte in this way is the next bigger common unit used in computers and has a bit of a special position of its own. A byte is just the combination of 8 bits together into one combined unit ranging from 0000 0000 to 1111 1111 , but we'll get further into the math behind binary later. The visual separation into two groups of 4 bits with a space between like in 0000 0000 is only to make it visually easier to read it compared to one giant 8-digit number.

This is also where we reach the next complications though. A single bit with the 2 values 0 and 1 is easy. But what number for example would the binary number 0010 1101 be? We don't have tens or hundreds positions/columns as we have in our regular decimal numbers that we're used to in everyday life. That's where we get to the binary number system.

The binary number system and how it works

As mentioned earlier already, the way we handle binary numbers is based on and works according to the same principle our everyday decimal numbers like 531 (five hundred thirty-one) also work. So let's get back to principles and first repeat how our decimal numbers work. Each position has a different inherent multiplier.

531

=> 1 + 30 + 500

=> 1 * 1 + 3 * 10 + 5 * 100

But how do we generalize the ones, tens, and hundreds position into a different number system like binary? We need to get a bit more detailed into how the positions in the decimal numbers work. For that, we number the position in the decimal number from the right to the left starting with the 0th position at the very right. So for 531 that would be 1 at the 0-position, 3 at the 1-position, and 5 at

the 2-position. Then going from that we multiply the value at every position with the base of the number system (10 in our decimal system) to the power of the position at which the digit appears in the number. For example, the 5 in 531 is worth $5 * 10^2$ or $5 * 100$. Let's make a more detailed example.

531

$$\Rightarrow 1 + 30 + 500$$

$$\Rightarrow 1 * 1 + 3 * 10 + 5 * 100$$

531

$$\Rightarrow 1 * 10^0 + 3 * 10^1 + 5 * 10^2$$

$$\Rightarrow 1 * 1 + 3 * 10 + 5 * 100$$

6792

$$\Rightarrow 2 * 10^0 + 9 * 10^1 + 7 * 10^2 + 6 * 10^3$$

$$\Rightarrow 2 * 1 + 9 * 10 + 7 * 100 + 6 * 1000$$

Converting binary numbers to decimal

The digit in the 0th position at the very right is easy since it's always the value that's written there times 1, no matter what number system we're using, since any number to the power of 0 always results in 1. But for the other positions, we'll have to redo it. So let's try the same example with a binary number. In binary we're calculating with base 2 instead of base 10 numbers, so we need to take the powers of 2 instead of powers of 10 for the calculations.

0010 1101

$$\Rightarrow 1 * 2^0 + 0 * 2^1 + 1 * 2^2 + 1 * 2^3 + 0 * 2^4 + 1 * 2^5 + 0 * 2^6 + 0 * 2^7$$

$$\Rightarrow 1 * 1 + 0 * 2 + 1 * 4 + 1 * 8 + 0 * 16 + 1 * 32 + 0 * 64 + 0 * 128$$

$$\Rightarrow 1 + 0 + 4 + 8 + 0 + 32 + 0 + 0$$

$$\Rightarrow 1 + 4 + 8 + 32$$

$$\Rightarrow 45$$

1010

$$\Rightarrow 0 * 2^0 + 1 * 2^1 + 0 * 2^2 + 1 * 2^3$$

$$\Rightarrow 0 * 1 + 1 * 2 + 0 * 4 + 1 * 8$$

$$\Rightarrow 0 + 2 + 0 + 8$$

$$\Rightarrow 2 + 8$$

```
=> 10
```

So the underlying principle is still the same and we can calculate the decimal value of long binary numbers, but you can tell how numbers become way longer when represented in binary and thus are way more inconvenient for calculations in everyday use. The way a computer works though with the dual state of on / off and translating that into bits with 0 and 1 means this is the simplest way to represent values in a computer, no matter how big. And thankfully most of the time we don't have conversions by hand. Many calculators can do that for you and even when doing low-level close to the metal programming you won't often have to deal with this.

Counting in Binary

Counting in binary (and really any number system) also uses the same principles as counting in decimal:

- You increase the value in the lowest position (very right position of the number) to the next higher value.
- If that value would overflow you reset it to 0 instead and increase the value in the next higher position too.
- If one of the higher positions also overflows you keep going further until there's no overflow anymore.

Let's use a 4-bit number as a counting example:

```
0000 => 0
0001 => 1
0010 => 2
0011 => 3
0100 => 4
0101 => 5
0110 => 6
0111 => 7
1000 => 8
1001 => 9
```

```
1010 => 10
1011 => 11
1100 => 12
1101 => 13
1110 => 14
1111 => 15
```

You might have also noticed another special relation to the values we can store and to the amount of bits we use. If you look above, we have 16 lines iterating the values from 0 to 15. That's 2^4 and $2^4 - 1$ respectively. In fact, a number with n bits is always able to represent 2^n values ranging from 0 to $2^n - 1$.

This also lets us count to way higher numbers with our hands. On one hand we can count to $2^5 - 1 \Rightarrow 32$ and with 2 hands we can count up to a maximum of $2^{10} - 1 \Rightarrow 1024$.

As a small example, what's the number I'm showing here?



Showing from right to left: 2 fingers extended, 2 fingers hidden, 1 finger extended

► The answer:

Conclusion

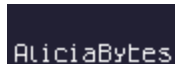
We've learned a lot about how bits and binary works here already, nice job! 🎉
With that said, there will still be plenty of times when those topics will come up again and you'll learn more about them, but for now it's really enough (this blog already got quite long 😊).

I'm planning to write more blog posts about IT and Computer Science from the ground up and turn them into a series, so feel free and follow along if you're interested in more.

Other cute beings and places you might want to check out:



And I also have an 88x31 over at [if you would like to add it.](#)



[Fediverse](#)

[Twitch](#)

[GitHub](#)

[YouTube](#)

[LinkedIn](#)

contact@aliciabytes.com

[About](#)

[Now](#)

[Recommendations](#)

[RSS](#)

[PGP](#)

[Privacy Policy](#)

[Cookie Policy](#)

[Functionality Overview](#)

[Colophon](#)

[88x31](#)